Planeamento e Gestão de Projeto

Etapa 1

GRUPO 08

Francisco Parasca - Nº 62233
Gonçalo Moutinho - Nº 62256
Martim Melro - Nº 62267
Miguel Lemos - Nº 62248
Rodrigo Sequeira - Nº 62255

Modelação

- *Diagrama de casos de uso*

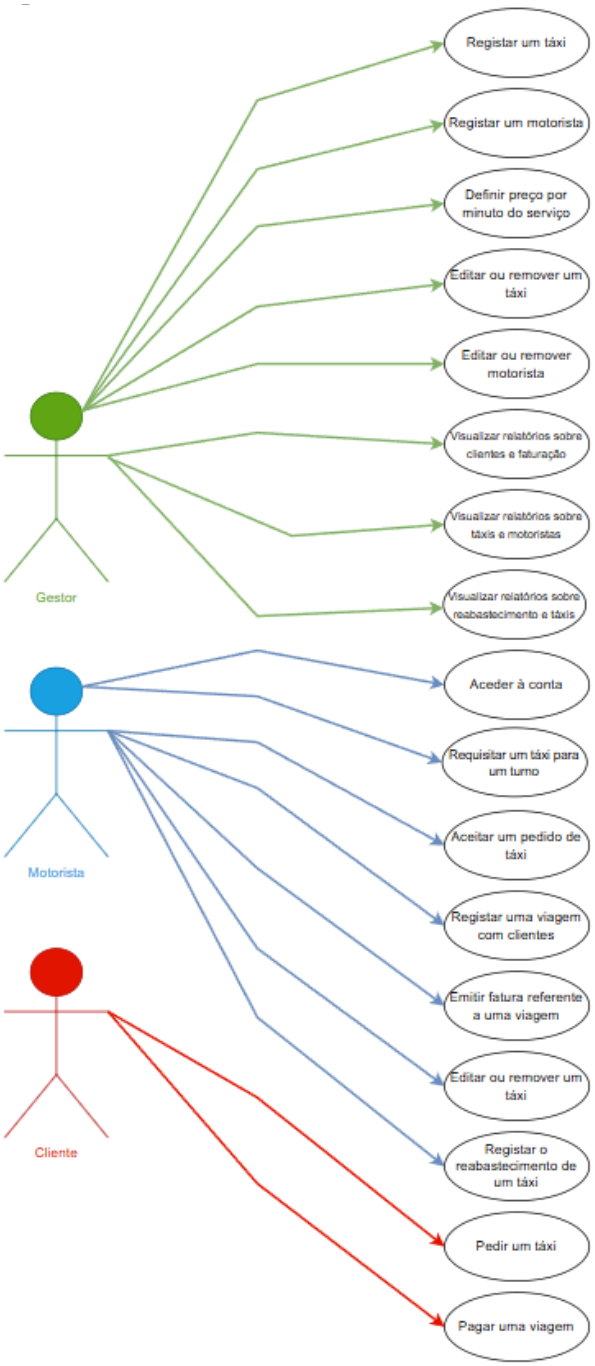
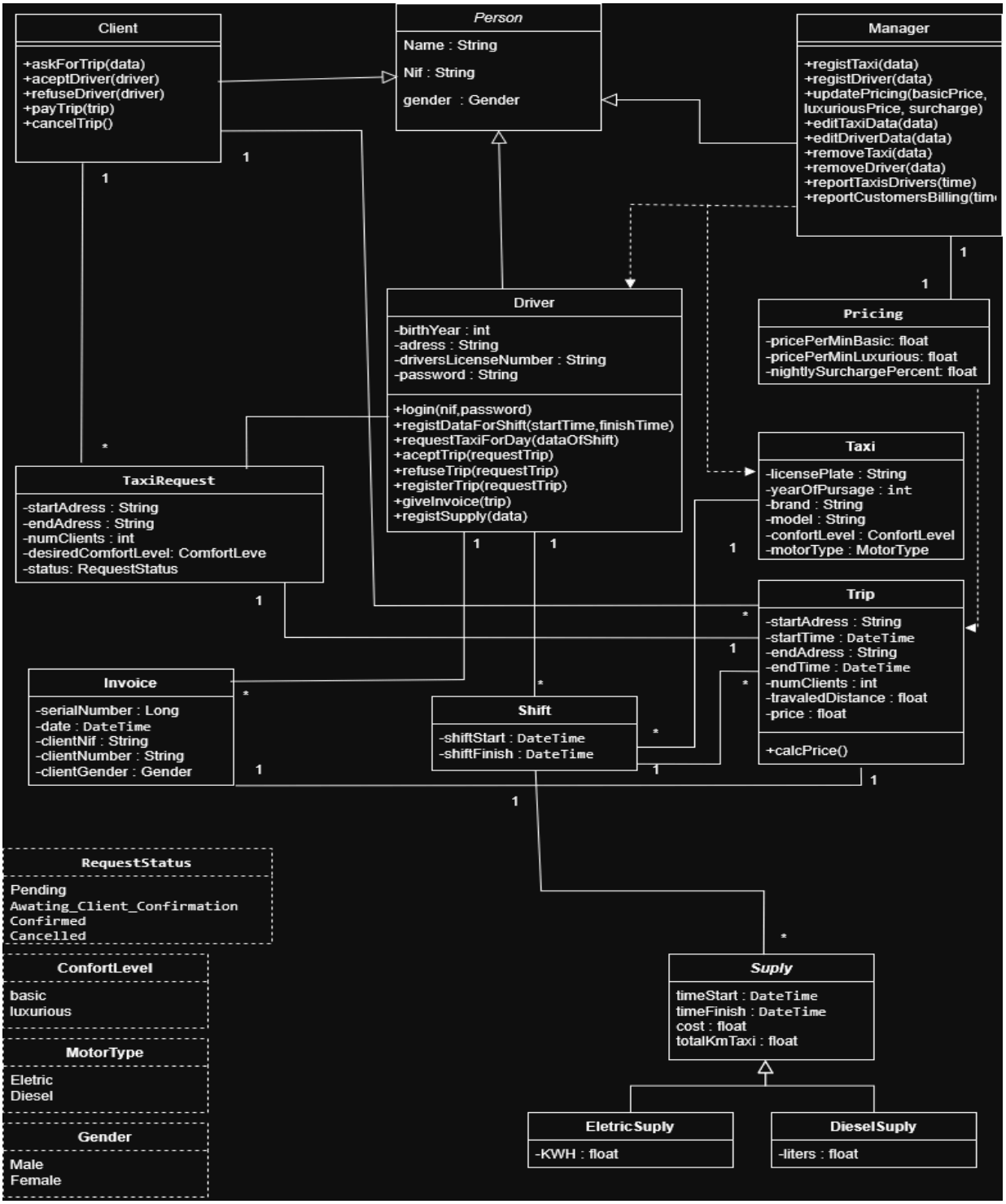


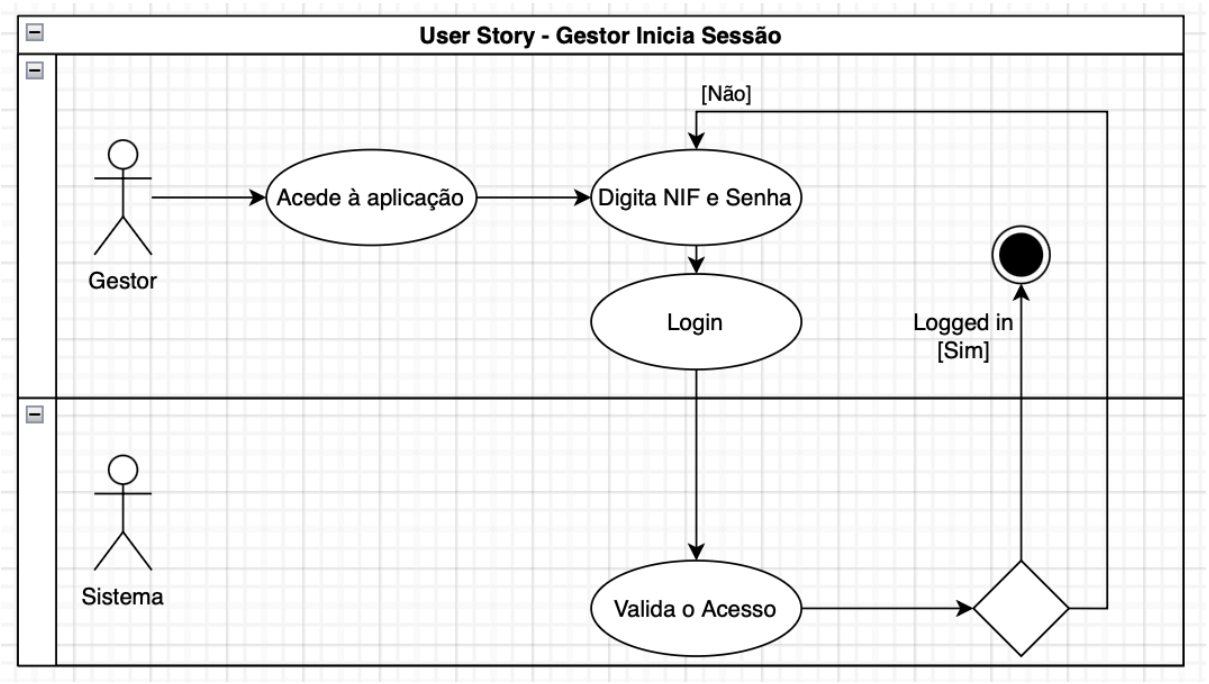
Diagrama que mostra quem interage com o sistema (os autores), neste caso o gestor, o motorista e o cliente, o que cada um tem a capacidade de fazer e de que forma é que essas interações se relacionam.

- *Diagrama de classes*



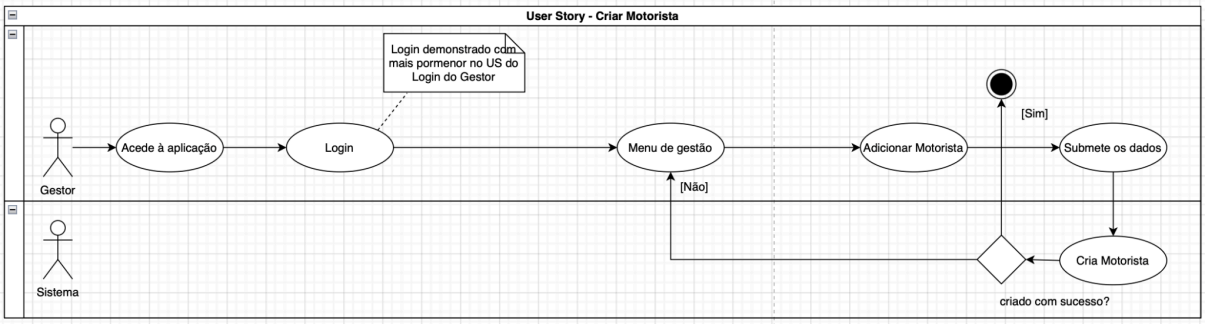
- *Diagrama de atividades*

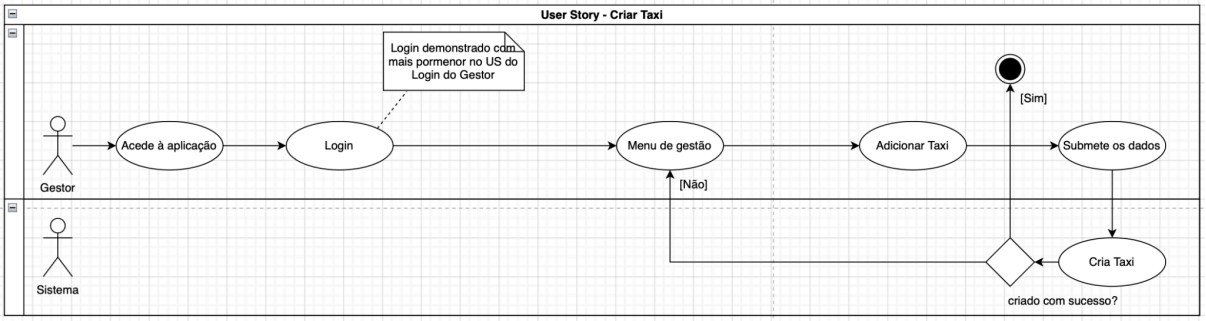
Diagrama de Atividade - Gestor Inicia Sessão (US Adicional)



Serve de apoio às User Stories relacionadas com gestores, é o login do gestor que tem de ser diferente do login normal porque não pode ter a forma de registar para manter a segurança.

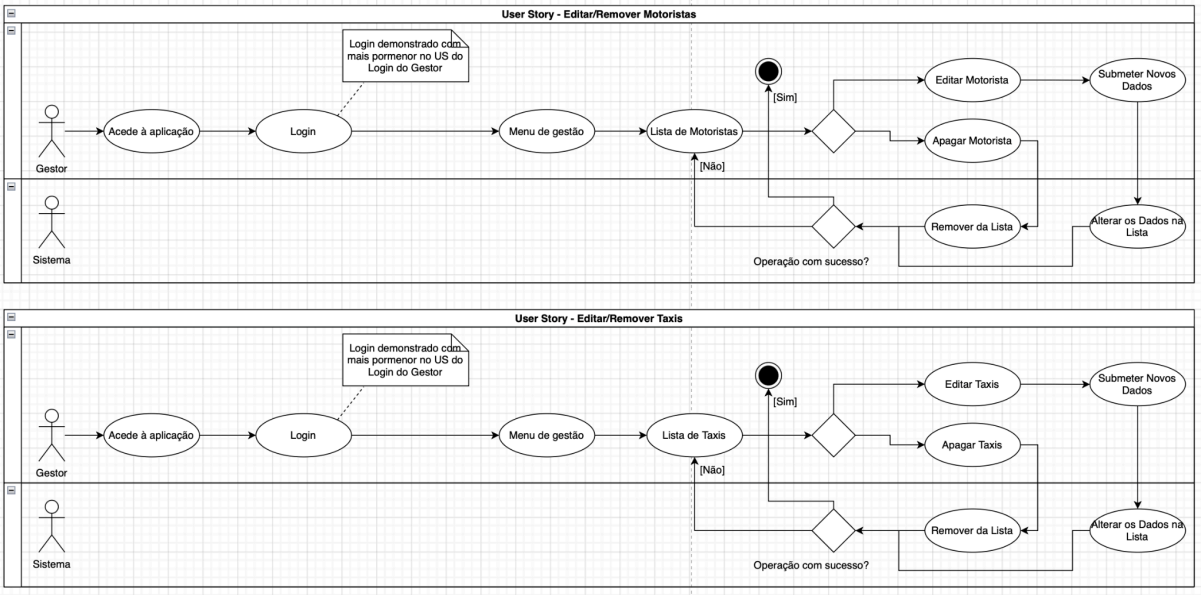
Diagrama de Atividade - Gestor a Registar Táxi ou Motorista (US 1 e 2)





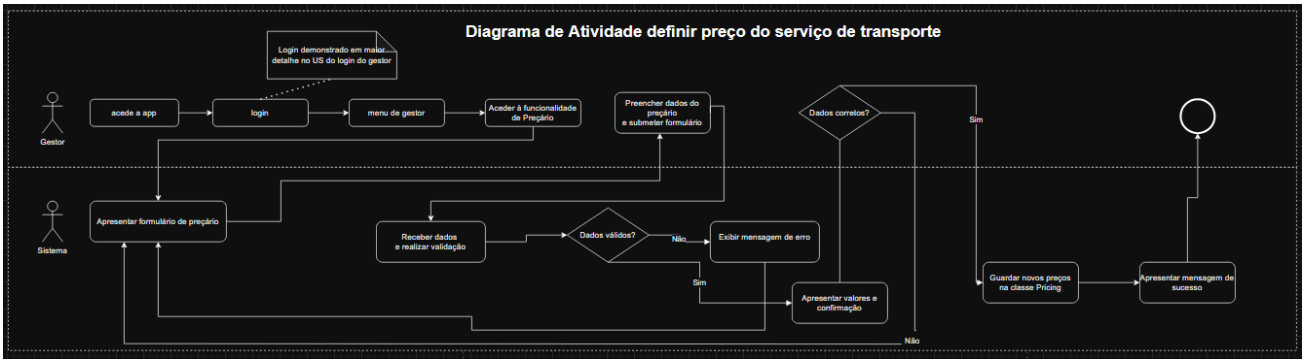
Ao fazer o login de acordo com o diagrama de apoio apresentado anteriormente é apresentado ao gestor um menu de gestor onde tem dois botões, um para adicionar um Motorista e outro para adicionar um Táxi. Após clicarem num dos botões é apresentado um formulário com os dados a serem fornecidos para a criação de Motorista/Táxi, todos eles com restrições impostas, ao submeter o formulário o sistema cria o Motorista/Táxi e acaba a atividade.

Diagrama de Atividades - Gestor a editar/remover Táxi ou Motorista - (US 12 e 13)



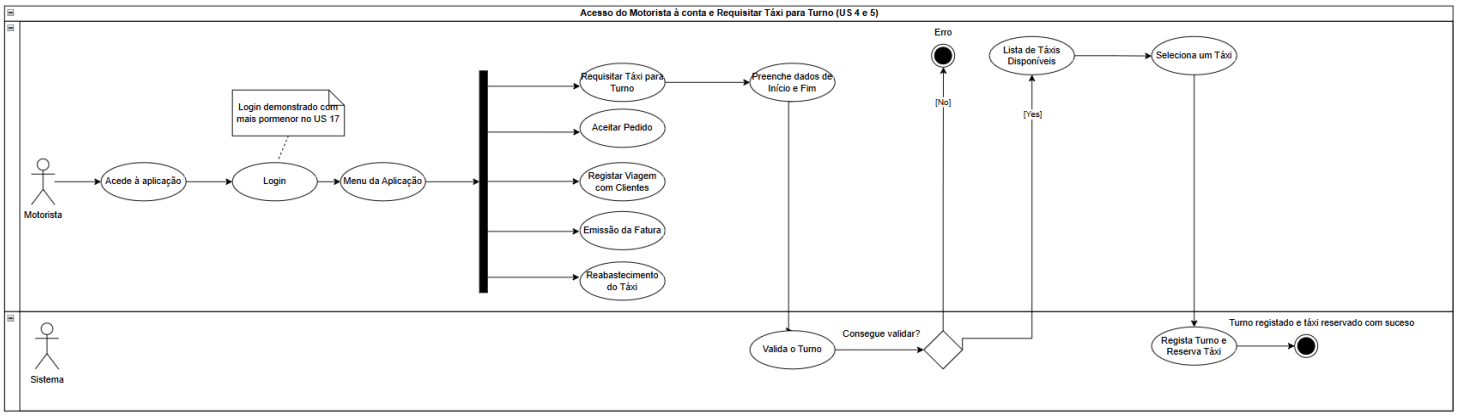
Volta a utilizar o login específico de gestor e apresenta uma lista de Motoristas e uma lista de Táxis. ambas ordenadas por ordem de criação. Dependendo da operação desejada faz-se o seguinte em cada lista, na linha do elemento desejado clicamos na opção desejada, seja essa apagar ou editar. Caso seja para apagar apenas se clica no botão e o elemento é apagado, caso seja editar a operação desejada é necessário preencher o formulário com os novos dados respeitando as restrições impostas. Após cada operação o sistema processa o pedido e finaliza a atividade.

Diagrama de Atividades - Gestor a definir preço do serviço de transporte - (US 3)



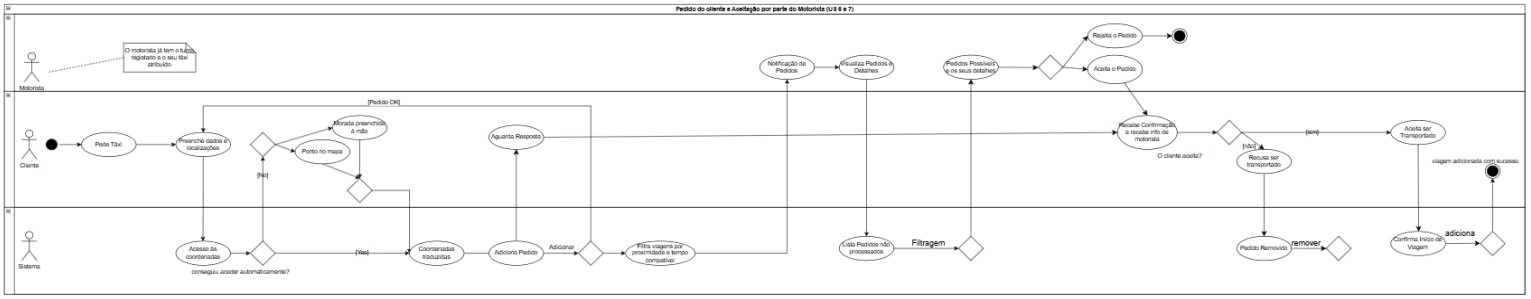
Após o login, o gestor acede à funcionalidade de Preçário no seu menu. O sistema apresenta o formulário, que o gestor preenche e submete. O sistema valida os dados; se forem inválidos, exibe um erro para correção. Se forem válidos, o sistema apresenta os valores para uma segunda confirmação pelo gestor. Caso o gestor não confirme, o processo regressa ao formulário. Se confirmar, o sistema guarda o novo preço na classe "Pricing" e finaliza a atividade com uma mensagem de sucesso.

Diagrama de Atividades - Acesso do Motorista à conta e Requisitar Táxi para Turno (US 4 e 5)



Este diagrama descreve o fluxo de trabalho do Motorista desde o momento em que tenta iniciar a aplicação até que um táxi é reservado com sucesso para um turno específico. O motorista inicia o processo fazendo login, após ser verificado pelo back-end, cumprindo as restrições 12 e 15, tem acesso ao menu da aplicação, onde seleciona a opção para requisitar táxi para um turno, o motorista escolhe as horas para o início e fim de turno, ocorre a validação do back-end (Restrição 6 e 2), se não for válido dá erro e é possível registar nova data, se for válido o sistema apresenta a lista válida de táxis disponíveis, o motorista escolhe e o back-end grava a reserva no sistema.

Diagrama de Atividades - Pedido do cliente e Aceitação por parte do Motorista (US 6 e 7)



Este diagrama descreve o fluxo de trabalho de correspondência entre um Cliente que solicita um táxi e um Motorista que aceita esse pedido, que culmina na confirmação da viagem. O processo está dividido em duas fases principais: a criação do pedido e a negociação.

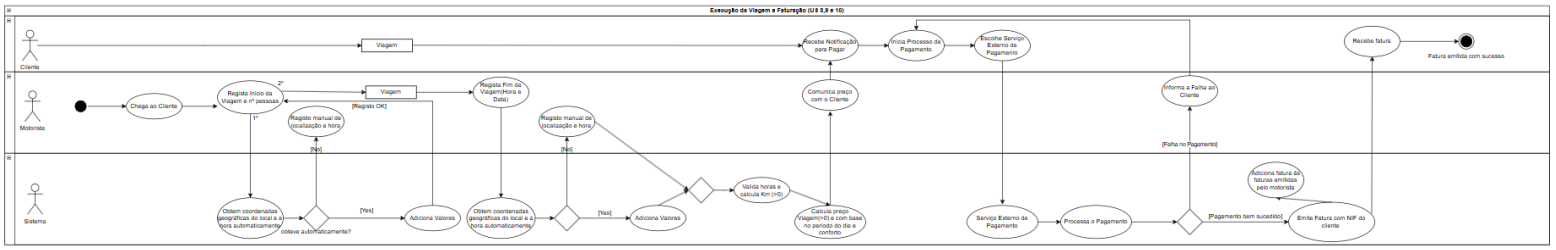
O processo inicia-se com o Cliente a preencher o formulário de pedido, indicando as suas moradas, o nível de conforto pretendido e o número de pessoas (Restrições 12, 13, 16 e 18). O Sistema tenta obter a localização do Cliente e do Destino através de coordenadas geográficas. Caso não a consiga obter automaticamente, o Cliente tem a opção de

introduzir as moradas manualmente, as quais são então traduzidas para coordenadas através de um Serviço Externo de geocodificação.

Após validar o pedido fica pendente, enquanto o back-end processa os pedidos pendentes, pois filtra os motoristas disponíveis e calcula a distância e o tempo estimado até ao Cliente.

O Motorista visualiza a lista de pedidos ordenada pela distância e decide qual aceita. Se aceitar o Sistema envia a proposta ao Cliente com a informação do motorista e o custo estimado da viagem. O cliente pode ou não aceitar o pedido, se aceitar a viagem começa, se recusar o sistema retira o pedido da lista do motorista.

Diagrama de Atividades - Execução da Viagem e Faturação (US 8,9 e 10)



Este diagrama descreve o fluxo de trabalho realizado pelo Motorista e pelo Sistema após a confirmação do pedido, que abrange o registo da viagem, o processamento do pagamento pelo Cliente e a emissão obrigatória da fatura.

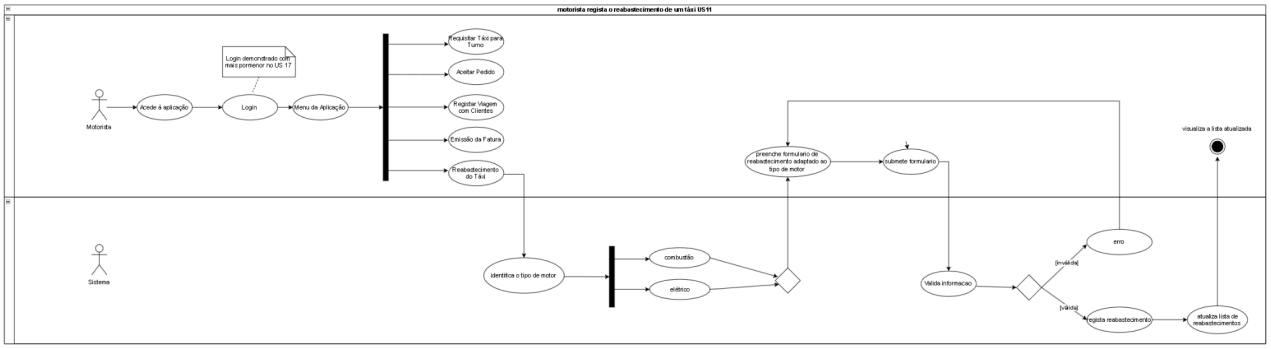
O fluxo começa com o Motorista a iniciar o registo do percurso (US 8). No momento em que o cliente entra no táxi, o Motorista regista o Início da Viagem (hora e morada de entrada), com o Sistema a tentar registar automaticamente a hora e as coordenadas geográficas. No destino, o Motorista regista o Fim da Viagem, e o Sistema deve registar automaticamente a hora e a morada de saída. O Back-end faz os cálculos e verificações necessárias, começando pela validação da coerência das horas (Restrição 1) e calculando os quilômetros percorridos (Restrição 19). Depois, processa o preço final com base na duração da viagem, o nível de conforto do táxi e o período do dia (Restrição 20). O Motorista comunica o preço final ao Cliente.

A fase de pagamento é realizada pelo Serviço Externo de Pagamentos (US 9). O Cliente escolhe o serviço e inicia o processo de pagamento. O Serviço Externo processa a transação e envia a resposta ao Back-end. Se o pagamento falhar, o Sistema informa o Motorista, e o Cliente é notificado para voltar a tentar o pagamento através de outro serviço online.

Se o pagamento for bem-sucedido, o Back-end emite a fatura sem introdução manual de dados pelo Motorista (US 10). O Sistema atribui o número sequencial da fatura (Restrição 21), valida a data (Restrição 8) e o NIF do Cliente (Restrição 12), e regista a fatura na base de dados, cumprindo o requisito legal. Com a viagem paga e com fatura, o processo é concluído.

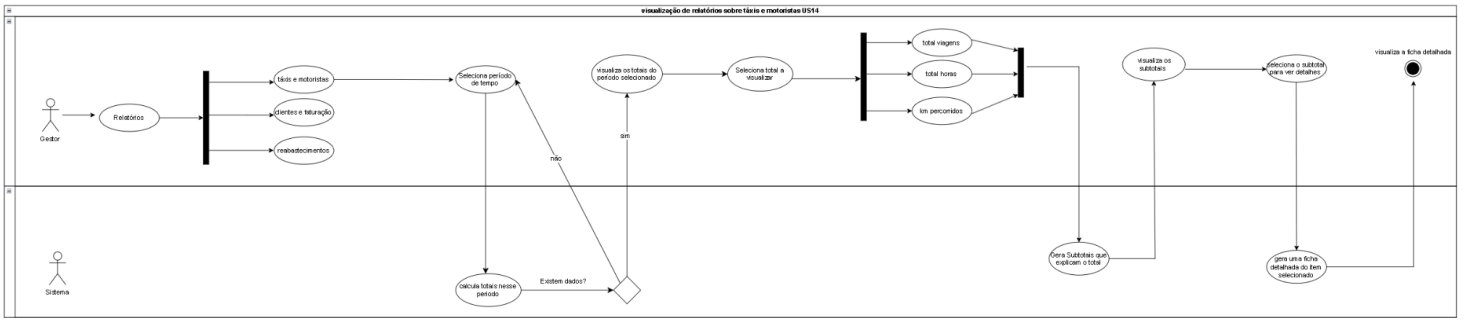
.

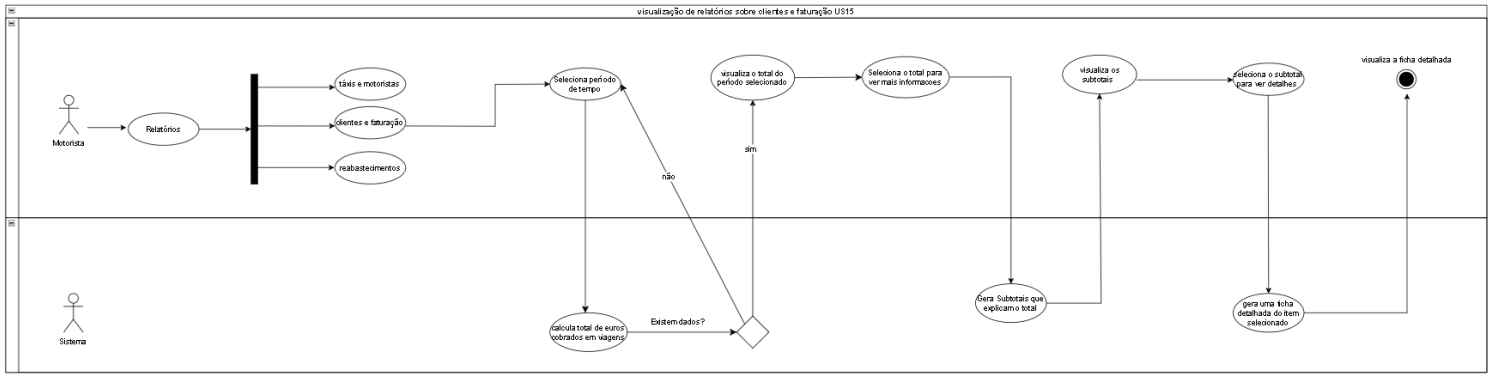
Diagrama de Atividades – Motorista regista o reabastecimento do táxi (US 11)

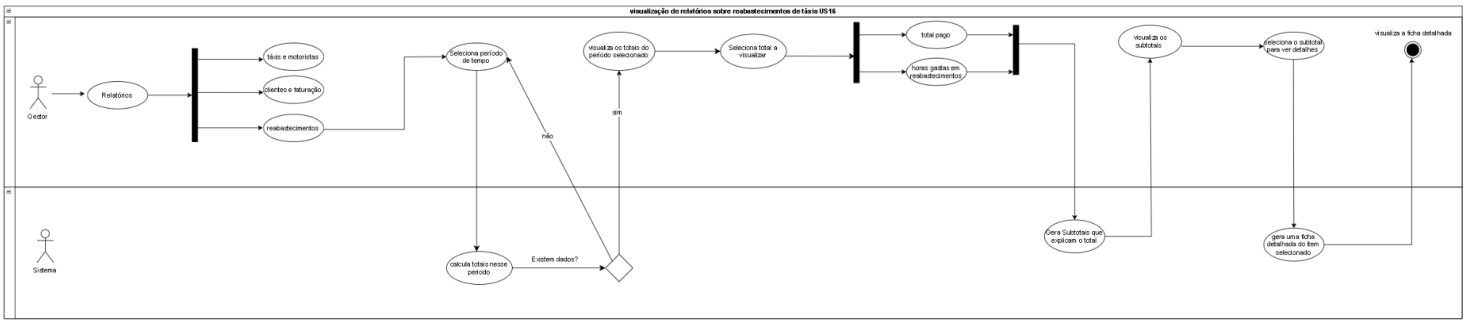


O processo inicia-se com o motorista a aceder ao menu da aplicação e a selecionar a opção “Registar Reabastecimento”. O sistema identifica automaticamente o táxi associado ao turno ativo e obtém o tipo de motor correspondente. Com base nisso, o sistema apresenta o formulário adequado.
O motorista preenche os dados do reabastecimento.
Após a submissão, o sistema valida as restrições definidas (restrições 1, 9, 10, 22 a 26).
Caso algum dado seja inválido, é apresentada uma mensagem de erro, permitindo ao motorista corrigir a informação.
Se os dados forem válidos, o sistema guarda o registo na base de dados e atualiza a lista de reabastecimentos do táxi, apresentando o novo no topo da lista, ordenada por data de forma descendente.

Diagrama de Atividades - Gestor a visualiza os diferentes relatórios (US 14, 15 e 16)







Estes diagramas de atividades(14, 15, 16) descrevem o processo de consulta de relatórios por parte do gestor, permitindo analisar o desempenho da plataforma em diferentes áreas:táxis e motoristas, clientes e faturação, e reabastecimentos e custos.
O fluxo inicia-se com o gestor a aceder ao menu de relatório, onde o sistema apresenta as três categorias disponíveis e após selecionar uma delas, o gestor pode através da indicação da data definir o período pretendido para a análise
O sistema valida os dados e consulta a base de dados para gerar os resultados correspondentes.O Gestor pode consultar detalhes mais específicos, como o total de viagens realizadas, o valor faturado, o consumo de combustível ou energia e o custo por quilômetro.

- *Diagrama de Implementação*

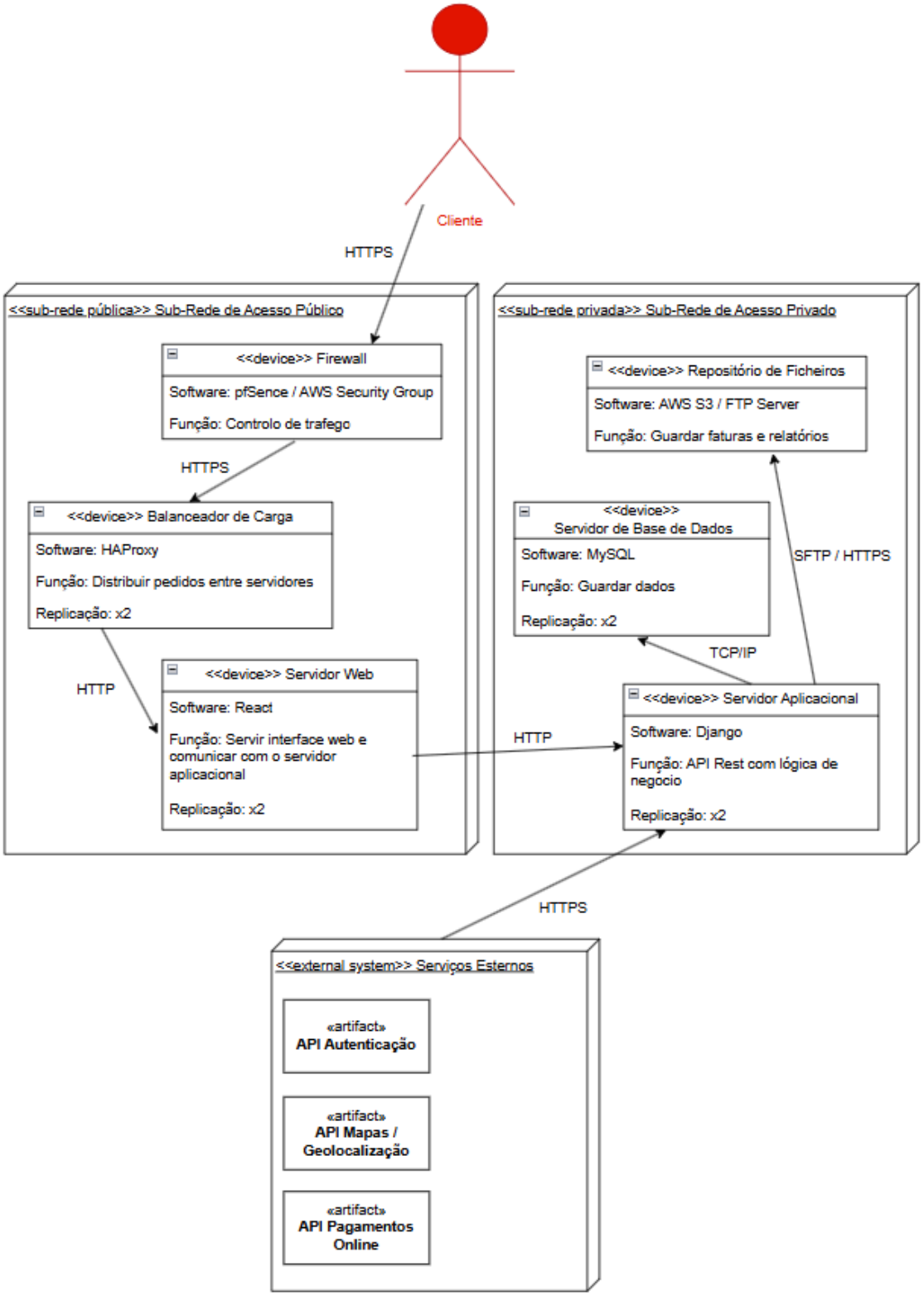


Diagrama UML que mostra como o programa está fisicamente instalado e distribuído, ou seja onde e em que servidores o software vai correr.

Pesquisa

- *Framework Frontend*

Para o desenvolvimento do frontEnd fizemos uma pesquisa das diferentes frameworks mais utilizadas na criação de interfaces web. Tivemos como objetivo encontrar a opção mais adequada às necessidades deste projeto, garantindo uma boa integração com o backEnd **Django**.

Entre as várias opções analisadas, foram consideradas quatro *frameworks* que se destacaram pela sua relevância para o projeto: **React.js**, **Vue.js**, **Angular** e **Svelte**.

O **React.js** é uma biblioteca JavaScript criada pela Meta(facebook) que é muito utilizada no desenvolvimento de interfaces baseadas em componentes reutilizáveis. É rápida, flexível e fácil de integrar com APIs REST.
Possui uma grande comunidade e várias ferramentas que permitem desenvolvimento de aplicações escaláveis. A utilização do Virtual Dom garante um desempenho eficiente e uma manutenção simples e organizada do código.

O **Vue.js** é uma framework leve e intuitiva utilizada em aplicações de pequena e média dimensão. A sua sintaxe é muito próxima do HTML tradicional o que facilita a sua aprendizagem. Consequentemente, como a sua curva de aprendizagem é reduzida facilita a adoção por equipas com menos experiência em Javascript.
Tem menos recursos disponíveis quando comparamos com a react.js, o que pode limitar a disponibilidade de recursos e bibliotecas.

A **angular** é uma framework super completo que foi desenvolvido pela google para aplicações de grande escala.É baseado em TypeScript, que Inclui várias funcionalidades como gestão de estado, routing, validação de formulários e testes automatizados.Contudo nem tudo é positivo pois ao ser um framework muito completo faz com que seja pesado e complexo de configurar o que não se adequa a projetos de menor dimensão como é o caso.

O **svelte** é uma framework que se diferencia das anteriores por compilar código no momento da build, eliminando a necessidade de um virtual Dom. Esta abordagem melhora o desempenho e reduz o tamanho final dos ficheiros.É bastante simples de aprender e permite criar interfaces de uma forma bastante rápida.Contudo, esta framework por ser bastante recente conta com uma comunidade de pessoas pequenas e menos bibliotecas disponíveis do que as outras.

Após a análise comparativa destas quatro opções de frameworks de FrontEnd, o nosso grupo optou pela utilização do React.js como tecnologia principal para o desenvolvimento da interface da aplicação.
Esta escolha deve-se à sua eficiente integração com o backend em Django, à facilidade de manutenção e à grande escalabilidade que proporciona.Para além disto, o React oferece uma estrutura que facilita o trabalho cooperativo permitindo que diferentes membros trabalhem em componentes distintos sem comprometer a coerência do projeto.
A sua grande comunidade e a extensa documentação disponível na internet também foi um factor que nos agradou bastante pois vai nos ser muito útil durante a execução do projeto.

- *Framework Backend*

Temos várias opções mas as mais relevantes são o ***Flask***, o ***Express.js***, o ***Django*** e a ***FastAPI***.

O ***Flask*** é uma framework mais minimalista e flexível perfeita para APIs e projetos de escala mais reduzida e tem o benefício de já ter sido usada no decorrer do nosso curso noutras cadeiras. No entanto dispõe de poucas ferramentas prontas e pode não ser adequado a projetos de maior escala.

O ***Express.js*** é uma framework minimalista e que é muito utilizada para o backend em ***Node.js***, é ideal para APIs REST. No entanto requer que fosse utilizado JavaScript que é uma linguagem que temos menos exposição no curso até ao momento atual e também apresenta um grande risco de desorganização do código em projetos de maior escala.

O ***Django*** é uma framework full-stack que dispõe de muitas ferramentas como ORM, autenticação, admin, entre outras. Tem uma estrutura bastante robusta, uma grande comunidade e um grande quantidade de documentação, no entanto é pouco

indicada para APIs de escala mais reduzida e tem uma curva de aprendizagem inicial bastante elevada.

A ***FastAPI*** é uma framework moderna ideal para APIs REST. É mais rápida e indicada a micro serviços, por isso mesmo, é também menos adequada a full-stack development.

O nosso grupo optou pela utilização do ***Django*** como framework ideal para o meu projeto.

- *Serviços Externos*

O nosso grupo decidiu a divisão dos serviços externos em 4 tópicos diferentes:

- Geolocalização/Mapeamento(Mapas, Coordenadas e Rotas)
- Autenticação de Utilizadores
- Pagamentos Online
- Tradução de Código Postal

Começando pela **Geolocalização/Mapeamento** necessária para o User Story 6 - Pedir um Táxi

Para a Visualização de Mapas centramos-nos em duas opções **Leaflet.js** e **Google Maps JavaScript API.**

O **Leaflet.js** é uma biblioteca JavaScript de código fonte aberto e gratuito, ideal para cumprir o RNF-04 e o RNF-14. É leve e simples, fácil de integrar e rápido para renderizar mapas básicos. No entanto, precisa de plugins ou de integração com serviços externos para funcionalidades avançadas.

O **Google Maps JavaScript API** é bastante abrangente e integra-se muito bem com os serviços de Geocodificação e Rotas do Google. Os seus dados são extremamente fiáveis e atualizados globalmente. Porém, além de ser mais lento a carregar que o **Leaflet.js** requer uma chave de API e o seu uso é pago após um limite.

Tendo em conta o RNF-04 e o RNF-14 e uma vez que já utilizamos o **Leaflet.js** numa unidade curricular anterior, optámos por lhe dar prioridade no uso deste projeto em relação ao **Google Maps JavaScript API.**

Quanto à Geocodificação a escolha recai entre duas opções: **Nominatim** e **Google Geocoding API.**

A **Nominatim** é um serviço de geocodificação de fonte aberta e gratuito.Utiliza Geocodificação Reversa e é bastante utilizada com o **Leaflet.js** ajudando a nossa escolha quanto a este tópico**.** Como desvantagem podemos apontar que possui restrições de uso mais restritas e que não é projetado para alto tráfego.

A **Google Geocoding API** além de ter uma fiabilidade e escalabilidade muito alta e precisão em traduzir moradas completas, é um serviço que é pago após o limite gratuito e que cria uma dependência do ecossistema Google.

Mais uma vez dando prioridade à opção que gera menos custo e já usada numa unidade curricular anterior, a nossa escolha é o **Nominatim**.
Para o Cálculo de Rotas e Tempos as duas escolhas seriam a **OSMR (Open Source Routing Machine)** ou a **Google Directions API.**

A **OSMR (Open Source Routing Machine)** é de fonte aberta e sem custos pode ser instalada em servidor próprio para controlo total, sendo apenas preciso chamar a API. Além disso, fornece rotas que considera estradas, sendo mais preciso que o cálculo Haversine (em linha reta).Porém, requer uma infraestrutura própria e a manutenção é mais complexa que a **Google Directions API.**

A **Google Directions API** é uma solução plug-and-play que retorna rotas, distância e tempo estimado de forma fiável e com alta precisão, em tempo real. É possível usar a aproximação dada pelo enunciado do projeto de 4 minutos para percorrer 1 km. Em contrapartida, é um serviço pago por chamada API.

Por essa razão numa primeira instância vamos utilizar o **OSRM,** porém existe a possibilidade de implementar a fórmula Haversine para a distância em linha reta como fallback.

Passando agora para a **Autenticação** que segundo o enunciado devem ser geridas pelo front-end (RNF-01), porém achamos que um serviço externo facilita a segurança.

Por essa razão e com a nossa pesquisa chegamos a duas conclusões: **Firebase Authentication (Google)** ou **Auth0.**

A **Firebase Authentication (Google) Firebase Authentication (Google)** apresenta um plano gratuito e escalável. Trata da gestão de tokens e hashes de senhas. A única desvantagem é que cria dependência do ecossistema Google.

Quanto ao **Auth0** é flexível e seguro, focado em grandes empresas e na segurança. Possui uma API REST completa e lida com normas de conformidade regulatória (RNF-15). Em contrapartida, o plano gratuito é limitado em termos de utilizadores e pode ser demasiado complexo para este projeto.

Por isso, para o nosso projeto vamos optar pela **Firebase Authentication.**

Agora para a parte dos Pagamentos a nossa pesquisa levou-nos a 3 opções: **Stripe, PayPal Braintree** e **MBWay**

Começando pelo **Stripe,** possui uma API REST de fácil integração e com bastante documentação, que cumpre o RNF-08. Oferece métodos de pagamento globais, carteiras digitais e checkout integrado. Minimiza também a responsabilidade da aplicação em relação aos dados sensíveis do cartão, o que apoia o RNF-11.No entanto cobra taxas de transação por percentagem e valor fixo, porém todas as opções cobram por este tipo de serviços. Outra desvantagem é a dependência total da API do Stripe.

Avançando para o **PayPal Braintree,** esta opção tem maior vantagem do que as outras duas por ser uma opção mais confiável para os utilizadores que já estão habituados a usar o serviço. É flexível e suporta múltiplos métodos de pagamento. Em relação à complexidade é maior em comparação com o **Stripe** e também cobra taxas adicionais.

Por fim, a última escolha foi o **MBWay** por ser uma opção local, e por ser atualmente o método de pagamento mais comum em Portugal. Oferece uma experiência de pagamento muito rápida para os clientes portugueses. Porém requer integração com o serviço SIBS/MB WAY, que não tem uma API de fácil acesso e é mais complexa que o **Stripe**, é limitado só ao mercado português mas cremos que isso não seja um problema para a realização deste projeto.

Contudo, a nossa escolha recai para o **Stripe,** sendo a escolha mais sólida. sua API REST fácil de integrar e uma documentação que é extensa facilita a sua implementação.

Quanto à Tradução de Código Postal para Localidade recaiu em duas: **Integração com Dataset de Código Aberto (GitHub)** dado no enunciado do projeto e **CTT API**

O **Dataset de Código Aberto (GitHub)** dado no enunciado é gratuito e de fonte aberta e além de ser uma pesquisa local e rápida cumpre o RNF-07 e não depende de API externas. Como desvantagens é de notar a coerência dos dados depende da frequência de atualização do dataset, não sendo em tempo real.

Já o **CTT API** garante que os dados do código postal são os mais atuais e oficiais em Portugal, pois é gerido pelo operador oficial. Porém o acesso a esta ferramenta dos CTT implica custos e não é de código fonte aberto.

Por essa razão optamos por utilizar o **Dataset de Código Aberto (GitHub)** dado no enunciado.

- *Sistemas operativos*

Os sistemas operativos que serão usados vão ser o ***Windows 11***, o ***Ubuntu*** e o ***MacOS Tahoe***, todos eles com a sua utilidade.

O ***Windows 11*** é o sistema operativo mais comum com bastante compatibilidade com o maior tipo de aplicações disponíveis no mercado mas tem tendência a consumir demasiados recursos em máquinas mais fracas.

O ***MacOS Tahoe*** é um sistema operativo que é limitado a computadores da marca ***Apple*** e tem alguns problemas com compatibilidade com algumas aplicações que não tenham sido desenvolvidas para a arquitetura dos processadores ***Apple*** ou para o sistema operativo em si.

O ***Ubuntu*** é uma distribuição do ***Linux*** e é sem dúvida o sistema operativo que menos consome recursos destes 3 aqui apresentados. Consome poucos recursos é compatível com tudo o que será necessário para o projeto mas é também mais centrado na linha de comando (**CLI**) o que aumenta a dificuldade do seu uso para utilizadores menos experientes com o sistema operativo.

O nosso sistema operativo de escolha foi o ***Ubuntu***.

- *Base de Dados*

Existem muitas SGDBs distintas e todas elas com os seus pontos fortes e fracos, mas as duas mais utilizadas são a ***MongoDB*** e o ***MySQL***.

O ***MongoDB*** é uma SGDB do tipo noSQL e que nós temos pouca prática na duração do curso, é bastante usada no desenvolvimento web mas tem os seus problemas com gerir consistência, integridade de dados e operações complexas.

O ***MySQL*** é uma SGBD do tipo relacional e é a que mais foi utilizada no decorrer do nosso curso, é simples e tem uma grande comunidade que facilita na procura de ajuda, é bastante usada no desenvolvimento web mas costuma ter mais dificuldades quando encarregue de grandes cargas.

O nosso SGDB de escolha é o MySQL que é simples, adequado ao nosso projeto e já foi utilizado por nós no decorrer do curso.

- *Balanceadores de Carga*

O projeto exige que o Balanceador de Carga seja configurado em modo Ativo-Passivo (High Availability), como indica o enunciado de PTR do ano passado, para evitar um ponto único de falha. Por essa razão, a nossa pesquisa levou a dois softwares: **HAProxy** e o **Nginx**.

O **HAProxy** é um software dedicado e otimizado para o Balanceamento de Carga e *proxy* de alta performance. É conhecido pela sua capacidade de lidar com um grande volume de tráfego com baixa latência (RNF-07). Enquanto o **Nginx** é um servidor web versátil, o **HAProxy** oferece mecanismos de *health check* e algoritmos de balanceamento mais avançados.

Ambas as opções são gratuitas e de código aberto, cumprindo o RNF-14 e RNF-04.

O facto de o **HAProxy** ter sido mencionado na unidade curricular de Projeto de Tecnologias de Redes sugere que a sua configuração está alinhada com as expectativas de implementação do projeto.

Por essa razão, a nossa escolha recai para o **HAProxy** que é reconhecido na indústria como a solução com melhor desempenho para balanceamento de carga e alta disponibilidade.

- *Balanceadores de Carga*

Para escolhermos o repositório de ficheiros, analisámos várias das opções mais populares do mercado, nomeadamente a **Google Drive API**, **o Amazon S3 (AWS)**, o **Google Cloud Storage (GCS)**, o **Azure Blob Storage** e o **Firebase Storage**. O peso foi dado a fatores como a escalabilidade, a facilidade de integração e os limites gratuitos.

Google Cloud Storage: Apresenta uma implementação bastante simplificada com o ecossistema da Google e oferece um desempenho bastante elevado, sendo uma ótima escolha para aplicações que utilizem outros serviços da Google, como é o exemplo o Maps. No entanto, o plano gratuito é precário em comparação com a concorrência e a configuração é mais complexa caso não se utilizem os serviços da Google.

Firebase Storage: É o mais simples de utilizar e possui integração direta com o Firebase Authentication, o que facilita a gestão de utilizadores. Apesar disso, pode levar a uma dependência do ecossistema da Google.

Azure Blob Storage: É a opção da Microsoft, ideal para projetos que utilizem infraestrutura Azure. Oferece um bom desempenho, segurança e integração com outros serviços da plataforma, mas é menos intuitivo e mais voltado para ambientes empresariais de grande escala.

Amazon S3 (AWS): É considerado o padrão da indústria para armazenamento de ficheiros na cloud, devido à sua elevada estabilidade, escalabilidade e segurança. Integra-se de forma direta com o Django através da biblioteca django-storages e disponibiliza SDKs para várias linguagens. A AWS oferece um Free Tier que inclui 5 GB de armazenamento, 20.000 pedidos GET e 2.000 pedidos PUT mensais gratuitos durante um período de 12 meses, o que é mais do que suficiente para projetos de pequena dimensão, como é o caso do projeto da cadeira.

Supabase Storage: É uma alternativa open-source que permite uma integração fácil com bases de dados PostgreSQL e aplicações web. Contudo, apresenta ainda algumas limitações em termos de escalabilidade e maturidade da infraestrutura quando comparada com as soluções mais consolidadas.

Tendo em conta que o nosso projeto é de pequena dimensão e não deverá ultrapassar estes limites, optámos pela utilização do **Amazon S3 (AWS)** como repositório de ficheiros. A sua facilidade de implementação com o Django, o plano gratuito suficientemente abrangente e o facto de ser a solução mais utilizada no mercado tornam-na a escolha mais adequada para as necessidades do nosso projeto.

Planeamento

*O planeamento foi enviado num ficheiro pdf à parte, que se encontra na pasta zip